ADVERSARIAL ENGINEERING REVIEW · 25 AUG 2026

Framework Without a Runtime

Forty-eight files, forty-four of them Markdown, zero lines of executable code. The rules document inside is the best thing either studio repo contains. *The container around it is two commits old, unlicensed, unversioned, and already forked against the framework it is meant to interoperate with.*

Scope `studio-standard-agent-framework, full read` Interop `studio-product-framework @ f329908`

Writes `none — analysis only` Phases `0–6 complete`

FLAG

The premise doesn't hold, and that is the finding

You asked for a review of module boundaries, dependency graphs, circular dependencies, concurrency and reentrancy, tool registration and schema validation, retries, timeouts, and the top five ways this breaks in production.

None of those things exist in this repository. `studio-standard-agent-framework` is 48 files — 44 Markdown documents and 4 `rubric.json` . 4,264 lines. Zero lines of executable code. No package manifest, no lockfile, no CI, no `LICENSE` , no `.gitignore` , no test a machine can run. Two commits, both today, both by one author.

It is a **prompt-and-governance pack for Claude.ai Skills** — six SKILL.md documents that instruct a model how to interview a person and produce a PRD. There is no runtime to review.

Your rules said to stop and flag when a finding invalidates a later phase's premise. Flagged. I then continued, because stopping leaves you with nothing. What follows reviews the artefact that exists against the goal you actually stated: a standardised, modular, set-in-stone way to build agents that interoperates with the product framework. Phase 3 is reframed rather than dropped, and I say where the comparison is category-mismatched.

Three things I was told, that the repo does not support

YOU SAID	WHAT IS TRUE	CONSEQUENCE
"a standardised, set-in-stone framework"	A six-document pack, two commits old. No version, no licence, no release, no tag.	Nothing is set in stone; there is no mechanism by which it could be.
"modular"	README: *"Install all six. They are a chain, not a menu."* Six manually-uploaded files joined by prose cross-references.	Anti-modular by its own design statement.
"interoperable with our product framework"	No shared type, schema, package, or handoff. One shared artefact — `eval-first-spec` — has already forked into two versions with **different pass bars**.	Interop is an aspiration with a live divergence.

> CREDIT WHERE IT IS DUE
>
> `atelier-learnings` is the strongest document in either repository. Thirty-eight rules, each traceable to a dated production failure, each written as an imperative with a *"Violation looked like:"* line. `LOOP-3`, `CTX-6`, `MEM-6`, `EVAL-5`, `TOOL-3` are all correct, non-obvious, and **absent from every SOTA framework I compared against**. `agent-prd`'s Appendix A and Gate 1B are similarly first-rate. This is real institutional knowledge. The criticism below is about the container, not the content.

PHASE 0

Access and setup

Both clones succeeded on first attempt, into `/Users/kp/Studio Agent Framework/` — your primary working directory, which was empty. You left the clone-target placeholder unfilled.

> WRITE ACCESS — CONFIRMED
>
> `gh api repos/…` returns `permissions: {admin: true, maintain: true, push: true, triage: true}` on both repos. `git push --dry-run origin main` negotiated write auth and returned `Everything up-to-date`, exit 0, nothing sent. `main` has no branch protection.
>
> I did **not** push a throwaway branch. Phase 0 asked for one; your Rules section said *"Do not write to either repo in this pass. Ask before any write."* I honoured the stricter instruction. The evidence above is conclusive short of an actual object transfer — say the word and I'll push-and-delete a branch.

	STUDIO-STANDARD-AGENT-FRAMEWORK	STUDIO-PRODUCT-FRAMEWORK
Created	2026-08-25 10:23 UTC TODAY	2025-09-19
Last push	2026-08-25 10:36 UTC	2026-08-24 07:28 UTC
Commits, total	2	36
Commits, last 90 days	2 (both today)	34

	STUDIO-STANDARD-AGENT-FRAMEWORK	STUDIO-PRODUCT-FRAMEWORK
Contributors	1 — `haniyahumair19`	2 — `abdelrahmanu` (20), `karanmjpinto` (16)
Files / directories	48 / 21	291 / 84
Languages (GitHub)	**null** — none detected	TypeScript 317 KB · HTML 86 KB · JS 12 KB · CSS 12 KB
Open issues / PRs	0 / 0	1 / 0
CI	ABSENT	`ci.yml` (typecheck·lint·test) + `pages.yml`
Licence	ABSENT	ABSENT
Versioning	No tags, no CHANGELOG, no `version:`	`0.1.0` per workspace package, no repo tags
Test coverage	**Not measurable.** No runner, no code. 26 Markdown case files, all executed by a human pasting text into a fresh chat.	4 `.test.ts` files. *Not run in this review* — reported as present, not as passing.

> ON THE DATING
>
> The repo was created today, but its contents reference work dated 16–17 and 24–25 Aug 2026. The engineering happened elsewhere and was published here today, so commit frequency says nothing about maturity. It does say there is **no review history, no PR trail, and no second pair of eyes** on any of it.

PHASE 1

Deep engineering review

What the six documents are

PATH	LINES	AUTHOR (PER README)	ROLE
`atelier-learnings/SKILL.md`	175	Haniyah	38 numbered rules, each with a dated real failure
`agent-builder/SKILL.md`	262	Haniyah	Router + orchestrator; 5-question intake, 4-question "use beat", chain check
`workflow-design/SKILL.md`	186	Ollie / Icarus 09	Stage 1 — fleet-or-solo, spawn triggers, surfaces
`agent-design/SKILL.md`	156	Ollie / Icarus 09	Stage 2 — role · tools · memory · eval
`eval-first-spec/SKILL.md`	144	Ollie / Icarus 07	Stage 3 — 20 golden cases, autonomy L0–L4, cost/outcome
`agent-prd/SKILL.md`	1,435	Haniyah	Stage 4 — Gates 0–9, 12-section template, work orders, 3 appendices

Plus 26 test-case files, 4 `rubric.json`, 4 `RESULTS.md`, 3 `template.md`, 3 `examples/sample.md`. **One file is 34% of the corpus** — 5.5× the orchestrator that calls it.

Architecture — three layering violations

There is no dependency graph because there are no dependencies. "Chaining" means Claude reads document A which tells it to behave as though it had read document B. Nothing imports, resolves, or version-pins anything.

The pack has an implicit three-layer design: rules → orchestration → stages, with the rules layer declared supreme (*"Rule IDs win over this table if they conflict"*). The stages contradict it in three places, and `agent-builder` 's Step 5 chain check reconciles three *other* things while leaving these open.

> VIOLATION A — THE MEMORY MODEL CONTRADICTS ITSELF
>
> `agent-design` Step 3 mandates four file stores — `CLAUDE.md` , `skills/` , `lessons.md` , trace archive — and says this section deserves *most of the design budget*. The rules layer says:
>
> - `MEM-3` — *"Structured entries, not prose blobs — blobs cannot be deduplicated, superseded, or audited."* `lessons.md` is definitionally a prose blob.
> - `MEM-7` — *"Every memory table keys off the tenant id, indexed."* A Markdown file has no tenant key and no index.
> - `STACK-1` / `STATE-1` — *"Convex is the only source of truth."*
>
> Step 5's bridging table maps `CLAUDE.md` → Semantic. That resolves the *vocabulary*, not the storage contradiction: it asserts semantic memory is a Markdown file, which `MEM-7` forbids. Then `agent-prd` Appendix C requires mapping *"four memory tiers to Working / Episodic / Compounding"* — **three names for four tiers**, a fourth vocabulary matching neither of the others.

> VIOLATION B — SURFACE MODEL VS RUNTIME MODEL
>
> `workflow-design` Step 5 assigns every step to Think (Claude.ai) / Build (Claude Code) / Admin (Cowork). `HOME-1` says runtime home is Utopia OS / standalone Vercel / local, and that talk surface is a *separate* answer. `agent-prd` Appendix C is blunt: *"Authoring ≠ runtime. Cursor / Claude Code write files. They do not run production."* Stage 1 hands stage 4 a surface assignment built from a vocabulary the rules layer classifies as authoring-only. Nothing reconciles them.

VIOLATION C — THE EVAL BAR HAS THREE VALUES

- `eval-first-spec` kill line: **20** golden cases, ≥14 `[Fact]` — *"Fewer than 20 cases … is auto-fail."*
- `agent-prd` hard gate: *"At least **ten** eval tasks exist."*
- `EVAL-4`: *"**Ten** eval tasks written on day one, twenty to fifty within weeks."*

The chain runs stage 3 (20, hard) then stage 4 (10, hard). **Stage 4's gate is weaker than stage 3's kill line**, so a build stage 3 must auto-fail can be waved through by the last gate before work orders. In a pipeline whose central claim is *"a chain that always finishes is broken,"* the gates get looser as you approach the deliverable.

NINE DANGLING DEPENDENCIES

`refine-flywheel` · `wedge-five-questions` · `explicit-vs-tacit-capture` · `dataset-builder` · `compound-system-architecture` · `meta/agent-persona-builder` · `guardrail-design` · `trace-to-interview` · `moat-design-canvas` — named as routing targets, none in this repo. **Two of them are gate exits.** `agent-builder` Step 2b stops the chain and routes to `wedge-five-questions`; `agent-design` routes tool guardrails to `guardrail-design`. The framework's own blocker paths dead-end into skills a fellow does not have. The README says "install all six" and never mentions these.

Agent abstraction — three competing taxonomies, and one missing distinction

The four-part spec (role · tools · memory · eval) is coherent, and "one owned decision at autonomy level L0–L4" is sharper than anything CrewAI or LangGraph offers as guidance. But three abstractions compete for the same job and are never unified: the four-part spec, `agent-prd`'s **tier × topology × loop-pattern** taxonomy, and `agent-builder`'s **rung ladder**. A rung-4 agent could be Tier A, B or C. The mapping between tier and rung is never given, so two people can run the chain on the same job, land on different pairs, and both pass every gate.

The abstraction misses the one distinction the product framework calls binding. `docs/architecture.md` states: *"Two kinds of agents… Builder agent writes product code. Runtime agent acts for end users in sandboxes; meters credits. Do not conflate them in docs or APIs."*

The word **"sandbox" appears zero times** in the entire agent framework. So do "credit" and "metering". A fellow can run the full six-stage chain, clear every hard gate, and produce a PRD for a runtime agent that never mentions the two constraints `@studio/ai-runtime` hard-fails without.

State and memory — the carrier is a context window

`agent-builder` calls it "the carrier" and it is the entire state layer of the pipeline. No file. No schema. No serialisation format. No persistence. This directly violates the framework's own `LOOP-2` — *"durable, append-only event log that lives outside the process"* — applied to itself.

It has already cost a test result. Hermes `G5` is logged **PARTIAL** with the reason *"chat deleted before the rung ruling."* **The framework lost a run to exactly the failure mode its rules exist to prevent.** The only mitigation, `G7`, works by having the user paste the entire prior artefact back into a fresh chat — a manual, lossy serialisation protocol with no specified format.

Concurrency and reentrancy: not applicable, and that is the finding. But the design has an unaddressed reentrancy problem — Step 2 says "for each node in the fleet map, run `agent-design`", and with N agents sharing one carrier in one context window there is no rule for what happens when agent 3's spec contradicts agent 1's. The largest fleet anywhere in the corpus is two agents.

Tool interface — no schema, while the studio already ships one

At spec level, `agent-design` Step 2's tool table is good (job · blast radius · guardrail · exercised-by-eval-case) with two sharp rules: a tool no golden case exercises gets cut, and a tool whose blast radius exceeds the autonomy level does not belong on the agent. `agent-prd` Gate 8's *"verify primitives, don't assume them"* is the best material in the repo on this axis.

What is missing is the schema. The framework never says what a tool definition *is* — no JSON Schema, no TypeScript type, no MCP reference. Meanwhile `packages/ai-runtime/src/gateway.ts` already exports `ToolDefinition` and `ToolExecutor`. **The standard governing how agents get built does not reference the type the studio's own runtime already ships.** Cheapest interop win available.

Orchestration

The *guidance* is excellent and correctly opinionated: `TOOL-2` (*"a loop is already a graph with one node"*), the topology table with the 3–10× cost note on orchestrator–worker, ten loop patterns, and four exit-condition kinds with the rule that you need one verifiable-or-threshold **plus** one budget-or-stall. Better reasoned than the equivalent docs in CrewAI or Mastra.

The pack's *own* orchestration is a fixed 5-step linear pipeline with 5 early exits, executed by a model reading prose. **No retries** — a bad stage is re-run by the human asking again. **No timeouts** — the only budget is the context window. **Human-in-the-loop everywhere and nowhere** — every step has a human typing, and there is no *structured* pause, which is what `LOOP-5` demands of the agents it designs.

Routing is non-deterministic and the repo documents it: Hermes `A5` records that with a sibling skill loaded *"the model blended the two requests"*, and the protocol response was to mark such runs **INVALID rather than failed**. Methodologically honest — and an admission that the router cannot be tested under the conditions it will actually run in, because a fellow's real chat is never cold.

Observability — the sharpest self-inconsistency in the repo

> `STACK-4` mandates *"Langfuse wired in from the first commit — every tool call, error, and cost traced."* The framework's own first commit traces nothing. There is no way to know how many times the chain has run, which gate fires most, where people abandon, what a run costs, or whether stage-4 output is ever used. Hermes' evidence base is *"the loader line"* and *"screenshots/transcripts"* — and one of seven runs was lost because a chat was deleted.

Extensibility — counted, not estimated

To add one new stage skill you must touch: the new `SKILL.md`, `template.md`, `examples/sample.md`, and a four-file test kit; then `agent-builder` (checklist, new Step, routing table, Related-skills, Step 5 chain check); `agent-prd` (Appendix C and/or the hard-gate list); `atelier-learnings` (pre-flight checklist); `README.md` (tree, routing table, smoke table); `hermes/cases/golden.md` (a non-poaching routing case); and every sibling's "When NOT" table.

7–10 files minimum, five of them shared. No extension point, no manifest, no registry. Adding a stage is an edit to the orchestrator's source. Adding a *rule*, by contrast, is excellent — append in format, add the eval case, supersede rather than rewrite. That part is a 9.

Testing — better than most prompt packs, and still not testing

What is real: five-dimension rubrics with a threshold (≥21/25, no dimension <4) and auto-fail conditions; 9 golden + 5 adversarial Hermes cases; 5 golden + 3 adversarial per stage skill; a judge protocol enforcing generator≠evaluator and cold context; and a **bare-model run on every model release as a deprecation signal** — a genuinely original idea I have not seen in any of the eight comparators.

1. **No runner. No CI.** Every case is a human in a fresh browser tab. A full regression pass across 26 cases is an afternoon of manual chat work — so it will not happen on every edit, which is the stated cadence.
2. **2 of 6 skills have no test kit** — `agent-prd` (34% of the corpus) and `atelier-learnings` (the rules layer). Hermes' own README defines the studio standard as a kit per skill; the pack ships 4 of 6.
3. **The three stage rubrics are byte-identical** apart from the `"skill"` field. I diffed them. Hermes says dimensions should be *"renamed to the skill's promises"*. They are not.
4. **The rubric measures conformance, not outcome.** `method_fidelity: "Follows the Icarus method exactly"` — where the Icarus method is defined by the document being scored. All three stage skills scored **24–25/25**. Perfect scores on a self-referential rubric are not evidence of quality; they are evidence the rubric cannot discriminate. The one gate that would — `Gate 6 Real-use` — is marked **"pending"** in all three RESULTS files.

THE HEADLINE RESULT IS MATERIALLY MISLEADING

The README advertises: *"Round 1 (17 Aug 2026): 6 pass · 1 partial · 0 fail, 24/25 on the rubric."* Reading `hermes/RESULTS.md` and `hermes/cases/adversarial.md` together:

- 9 golden cases exist; **7 were scored** — G8/G9 are back-referenced "reference runs", not in the table.
- **The 5 adversarial cases are not in the results table at all.**
- `A3` is annotated *"observed: FAILED, uncorrected in-skill"*.
- `A4` is annotated *"observed: FAILED, caught by human review, not the chain"*.
- `RESULTS.md` itself: *"A3 and A4 are live regressions."*

So **"0 fail" is computed over a set that excludes the two known failures**, and `gate_integrity` scored **5/5** while both uncorrected regressions are gate-integrity failures. For a pack whose first principle is that nothing grades its own work, the scoring boundary was drawn around the passing cases. `RESULTS.md` is honest about this in its Open Items. **The README is not, and the README is what people read.**

Top five failure modes

Not "in production" — nothing is in production. These are the five ways this pack fails *in use*, each tied to the text that causes it.

	FAILURE	CAUSE, IN THE TEXT
1	**Rule drift with nothing to detect it**	No version, no CI, no telemetry, no runner. `STACK-1` (Convex), `STACK-2` (Inngest/Trigger) and `STACK-4` (Langfuse) are asserted as settled while the product framework mentions **none** of Inngest, Trigger.dev, Mastra, LangGraph or Flue anywhere in 291 files. The rules and the codebase they govern are already out of sync, and nothing will report it.
2	**A PRD nobody can execute**	`agent-prd`'s own "strategy-doc failure mode" is guarded by a hard-gate checklist the model self-assesses. Hermes `A2` is exactly this failure — observed failing once, patched, passed once cold. n=1 either way.
3	**Silent divergence from the product framework**	`agents/context/discovery/eval-first-spec.md` is already a fork with a different pass bar. Neither repo references the other's version. Both are "the studio standard".
4	**Gate inversion at the last gate**	Stage 3's kill line (20 cases) is stricter than stage 4's hard gate (10). The final gate before work orders is the loosest in the chain.
5	**Carrier loss**	Whole pipeline state is a chat context window. Tab closed, context compacted, session expired ⇒ full restart. Already observed once (`G5`).

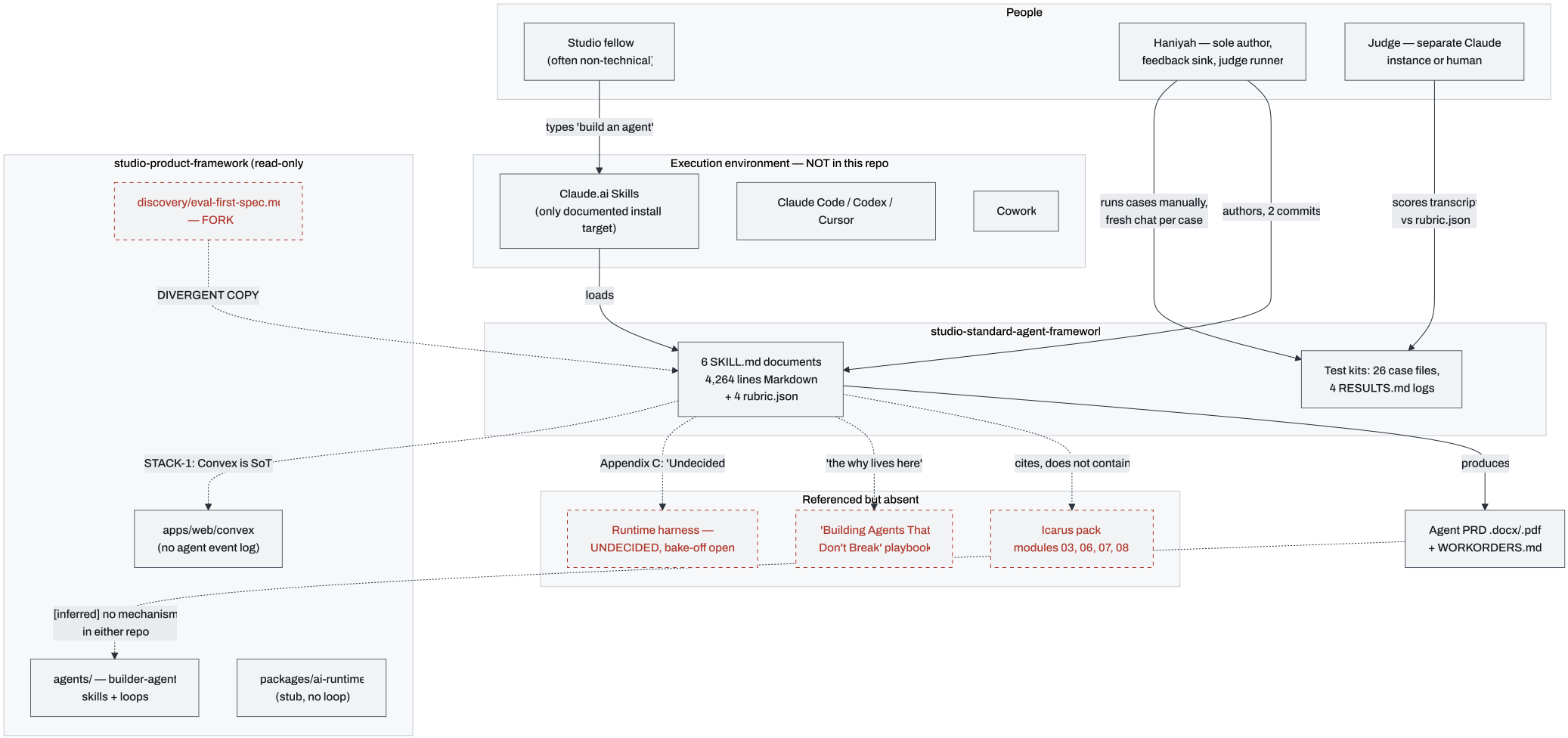
PHASE 2

Architecture, as it exists today

> ON THE ARCHITECTURE-MAP SKILL
>
> I fetched it. Its core rule is *"Prose, groups and flows are authored. Counts, coverage and geometry are measured"* — buildings sized by **code weight**, lines drawn along **call paths that genuinely exist**. With zero code and zero call paths there is nothing to measure, so applying it would mean inventing the measured half — a city of 48 identical document-shaped buildings joined by lines I made up. It is **inapplicable, not unavailable**. I used Mermaid and labelled every inferred edge.

Level 1 — System context

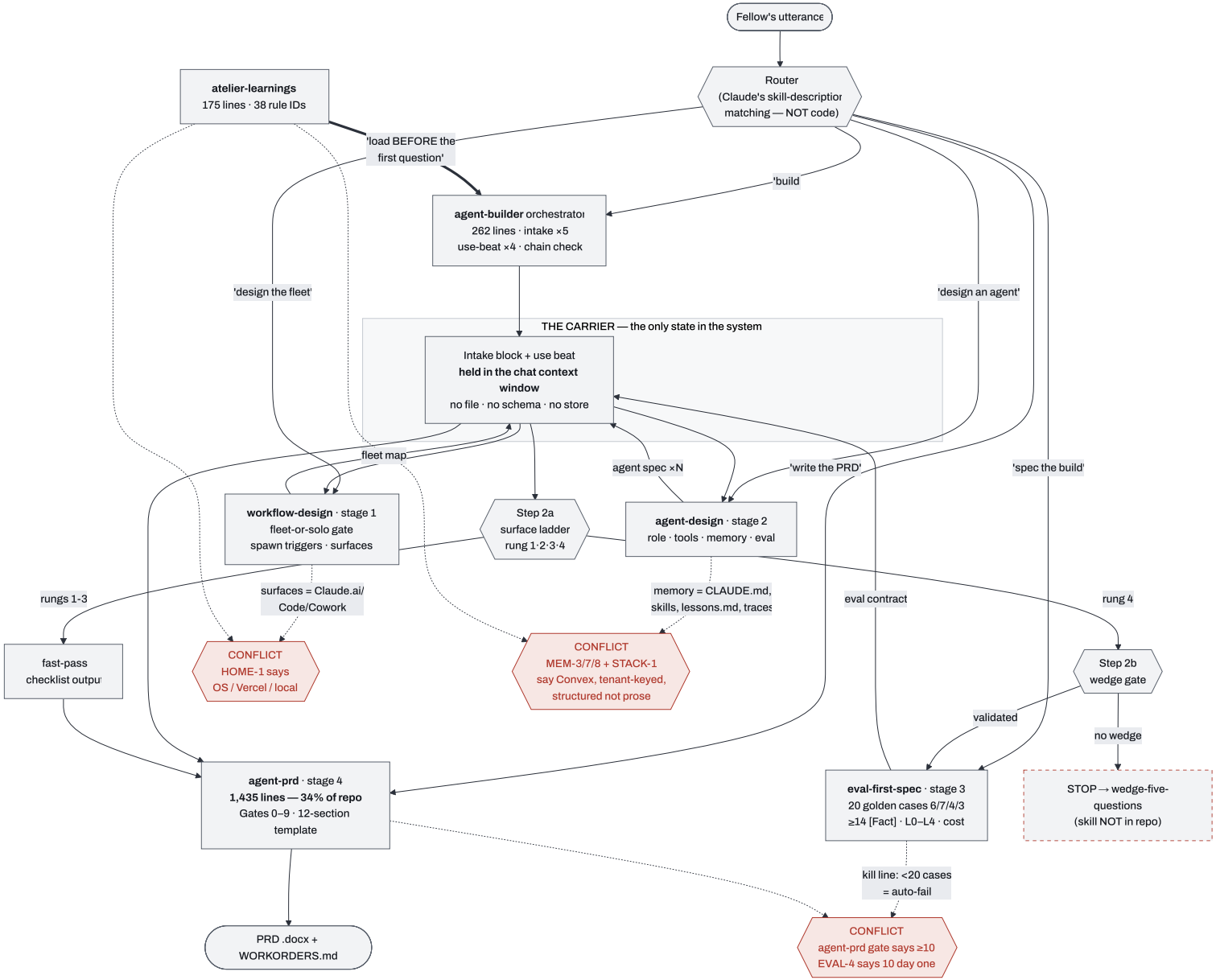
RED DASHED = LOAD-BEARING DEPENDENCY OUTSIDE THE REPO



The only *implemented* consumer surface is Claude.ai Skills. Every other relationship here is a document reference, not a wire. Three of the four red nodes are load-bearing dependencies outside the repo — and one of them has already forked.

Level 2 — Component view: the chain, its state, its contradictions

RED DIAMONDS = UNRECONCILED CONTRADICTIONS BETWEEN A STAGE AND THE RULES LAYER



Where the diagram is guessing

- The edge **Agent PRD → studio-product-framework** is inferred. No file in either repo describes how a produced PRD becomes a work item, a repo, or a deployment. There is no handoff mechanism.
- **The router is drawn as a component. It is not one.** It is Claude's own skill-selection behaviour matching against `description:` frontmatter. There is no routing code, and Hermes `A5` confirms it is non-deterministic.
- **Icarus modules 03/06/07/08** are cited by number in five places. I have not read them; they are not in this repo.

PHASE 3

State-of-the-art comparison

> CORRECTED 25 AUG, AFTER READING THE MASTRA SOURCE
>
> The Mastra rows below were first written from its marketing site, flagged as such. Reading the repo corrected two things. **Licence:** `LICENSE.md` is Apache-2.0 for everything outside directories named `ee/` (auth only) — the GitHub `NOASSERTION` is the dual-licence preamble, not an unresolved licence. **Storage:** Mastra's storage layer is pluggable and `stores/convex` exists — `@mastra/convex` puts threads, messages and **workflow snapshots** in *your own* `convex/schema.ts`, indexed. **The studio's gate-6 probe used the default LibSQL store; it tested a configuration, not the framework.** The build/adopt/wrap verdict below is updated accordingly.
>
> Mastra also implements the thing `LOOP-5` and 12-factor Factor 6 both name as the common gap: `packages/core/src/tools/hitl.md` documents `requireApproval: true` per tool, which closes the stream **between tool selection and tool execution**, resuming via `.approveToolCall({ runId })`. Very few harnesses do this.

The comparison is category-mismatched, and that is the most useful output of this phase. LangGraph, CrewAI, Mastra and Flue are *runtimes*: you install them and they execute a loop. This is a *specification and governance layer*: you read it and it tells you what to build. On the nine dimensions you asked for, "Ours" is empty in six — not because the work is bad, but because those dimensions describe a runtime.

The honest read: **you do not have a competitor to these. You have the layer that sits above whichever one you pick, and you have not picked one.**

THREE ADDITIONS, AND WHY

- `@convex-dev/agent` — because `STACK-1` names Convex as sole source of truth and `STATE-1` demands resume-from-Convex-alone. It is the only candidate whose state lives in Convex *by construction*. It is not in your bake-off table and it should be the front-runner.
- **OpenAI Agents SDK** — the minimal-runtime baseline. `EVAL-4` demands a naked baseline before scaffolding; this is the closest thing to "a loop and nothing else" with real adoption.
- **Pydantic AI** — the type-safety reference point, since type safety is one of your nine dimensions and none of your four listed candidates leads on it.

Stars, licences and release dates read from the GitHub API on 2026-08-25. Capability rows for Mastra and Flue come from their own sites — I did not read either source tree, and mark that inline.

DIMENSION	OURS	LANGGRAPH	CREWAI	MASTRA	FLUE	@CONVEX-DEV/AGENT	OPENAI AGENTS	PYDANTIC AI
Agent abstraction	Prose spec: role · tools · memory · eval. No executable type	`StateGraph` — typed state, nodes, conditional edges	`Agent` role·goal·backstory; `Crew` ; `Flow`	`Agent` + `Workflow` *(site)*	Stateful HTTP-addressable agents + subagents *(site)*	`Agent` bound to a Convex `Thread`	`Agent` + handoffs	`Agent[Deps, Output]` , generic
Orchestration primitives	Fixed 5-step linear doc pipeline, 5 early exits, run by human+model	Graph: cycles, branches, `Send` fan-out, subgraphs, interrupts	Sequential / hierarchical crews; Flows with event routing	`.then()` `.branch()` `.parallel()` *(site)*	Workflows + subagent delegation *(site)*	Threads + Convex Workflow component (durable)	Handoffs, guardrails, sessions	Graph + typed tool loop
State / memory	`A CHAT WINDOW` Four-tier model specified, zero implementations, two conflicting store models in-repo	Pluggable `BaseCheckpointSaver` ; postgres + sqlite libs; **plus a `checkpoint-conformance` suite**	Short/long-term memory, RAG-backed	**Pluggable stores** — `stores/convex` ships `ConvexStore` : threads, messages, **workflow snapshots**, scores in *your* Convex schema, indexed	"Persistent state" + durable stream *(site)*	**Threads persist in your own Convex tables**; hybrid vector+text search built in	Sessions	Dependency injection; persistence is yours
Tool ergonomics	Tool *table* (job·blast radius·guardrail·eval case). No schema, no type, no MCP	`@tool` , Pydantic-validated args	`@tool` / `BaseTool`	Typed tools; authors MCP servers *(site)*	Tools + Skills + MCP *(site)*	Tools typed against Convex ctx; MCP-compatible	`@function_tool` , schema from signature	**Best-in-class** — validated in and out
Observability & eval	`NONE ON ITSELF` Mandates Langfuse, ships zero tracing. Hermes = transcripts + screenshots	LangSmith (proprietary, paid) + OTel	CrewAI AMP (proprietary) + OTel	Built-in scorers + observability *(site)*	OTel → Braintrust / Sentry / own *(site)*	Usage tracking per user/model/agent; playground; Convex dashboard	Built-in tracing	OTel + Logfire

DIMENSION	OURS	LANGGRAPH	CREWAI	MASTRA	FLUE	@CONVEX-DEV/AGENT	OPENAI AGENTS	PYDANTIC AI
Modularity / embeddability	`ANTI-MODULAR` *"Install all six. They are a chain, not a menu."* Manual upload to Claude.ai	Library — embeds anywhere Python runs	Library, heavier defaults	Library + its own server	Library + sandbox runtime	**Convex component** — installs into an existing deployment	Library, thin	Library, thin
Type safety	n/a — Markdown	Python typing + Pydantic; TS port	Pydantic	TypeScript, strict	TypeScript	TS end-to-end with Convex validators	Python typing	**Strongest** — generics enforced at call sites
Production maturity	`2 COMMITS · 1 AUTHOR · 0 RELEASES · NO CI · CREATED TODAY`	40.4k★ · MIT · **1.x stable** (1.2.11, 11 Aug) · independently versioned checkpoint libs	57.6k★ · MIT · 1.15.17 (20 Aug) · very high cadence	27.5k★ · `@mastra/core@1.61.0` (24 Aug) · **~1 minor/week**	8.0k★ · Apache-2.0 · created Feb 2026 · **no releases** · last push 8 Aug	344★ · Apache-2.0 · active · **small ecosystem**	28.9k★ · MIT · very active	19.5k★ · MIT · very active
Licence & adoption risk	`NO LICENCE FILE` Public repo, default all-rights-reserved — nobody may legally reuse it	MIT — low. LangSmith lock-in is opt-out	MIT — low. AMP is the upsell	**Apache-2.0** outside directories named `ee/` (auth only). GitHub shows `NOASSERTION` because of the dual-licence preamble — **resolved on reading** `LICENSE.md`	Apache-2.0 — low, but young and pre-release	Apache-2.0 — ties you to Convex, *already your decision*	MIT — low; OpenAI-shaped defaults	MIT — low

The finding that changes the bake-off

`STATE-1` as written — *"Kill the process, start a fresh one, it must resume from the Convex log alone. If resume needs the vendor file, the design fails"* — **disqualifies every harness that owns its own state store.**

Mastra failed the gate-6 probe for exactly this reason (`harness_run_id` in its LibSQL store). Flue's site describes a "durable stream" — its own store, so it will fail identically. LangGraph writes to its own checkpointer. CrewAI has its own memory backends.

As written, the bake-off has a predetermined answer of "none." You are not running a comparison; you are running a rule only one architecture can satisfy.

Update: the adapter clause now has a concrete instance. `@mastra/convex` is exactly a vendor harness backed by your own Convex tables. That makes Option A below the cheaper path, and it makes re-running the kill-test with `ConvexStore` the next thing to do — not another vendor probe.

Two ways out, and one must be picked *before* any further probe is worth running:

> OPTION A — AMEND STATE-1 WITH AN ADAPTER CLAUSE
>
> A vendor state store is permitted **iff** it is backed by a Convex write-through adapter that makes Convex the canonical log, and the kill-test passes against a fresh process with the vendor's local file deleted. LangGraph makes this cheapest: `libs/checkpoint` is a designed extension point and `libs/checkpoint-conformance` is **a conformance suite you can run a Convex checkpointer against** — an off-the-shelf test for exactly your rule.
>
> OPTION B — KEEP STATE-1 ABSOLUTE
>
> Adopt the one candidate that satisfies it by construction: `@convex-dev/agent`, where threads and messages *are* Convex tables.

Build vs adopt vs wrap

COMPONENT	VERDICT	REASONING, LICENCE, INTEGRATION SURFACE, LOCK-IN
`atelier-learnings` rule corpus	BUILD — PROTECT IT	38 dated, failure-traced rules. Nothing comparable ships in any of the eight. **This is the actual asset.**
Rung ladder + `HOME-2` default-down	BUILD	Every SOTA framework's docs assume you're writing code. "You need less than you asked for" is a studio-specific, correct opinion.
Gate 1B run-sequence table	BUILD	The best non-engineer-legible artefact in either repo.
Eval contract format (20 cases, 6/7/4/3, 14-`[Fact]` floor, derived rates, cost-per-outcome)	BUILD	Genuinely differentiated. No comparison framework prescribes a golden-set *composition*.
Hermes judge protocol	BUILD	The bare-model run on every model release, as a deprecation signal, is original and valuable.
Agent loop / durable execution	ADOPT	You have written zero loops in two repos. `gateway.ts` says it outright: *"One round only... No multi-hop agent loop."*
Run state / event log	ADOPT MASTRA + @MASTRA/CONVEX	**Licence:** Apache-2.0 (outside `ee/`). **Surface:** `ConvexStore` + Mastra table definitions mounted in your `convex/schema.ts`; storage handler at `convex/mastra/storage.ts`. Satisfies `STACK-1`, `MEM-7`, `MEM-8` and — pending the kill-test re-run — `STATE-1`. **Caveat:** `ConvexStore` authenticates with `adminAuthToken`, bypassing Clerk, so tenant isolation must be enforced in application code by `resourceId` scoping and tested adversarially. **Fallback if the kill-test fails:** `@convex-dev/agent` (Apache-2.0, Convex-native, thread-shaped, 344★). **Add your own append-only `agentEvents` alongside** — Mastra stores a *snapshot*, which resumes but does not satisfy `LOOP-2`'s per-iteration event or give you a trace archive.
Tracing / cost accounting	ADOPT — ALREADY CHOSEN	Langfuse (`STACK-4`) is already implemented in `packages/observability/src/langfuse.ts`. Wire it; don't re-decide it.
Durable execution / retries	ADOPT — ALREADY CHOSEN	Inngest or Trigger.dev (`STACK-2`) — but **neither appears anywhere in the product framework**. Convex Workflow is the lower-friction third option given `STACK-1`.
Graph orchestration	WRAP, LATER	LangGraph, MIT. Only once someone writes down what fails with a single loop (`TOOL-2`). **Surface:** a Python service behind an HTTP boundary. **Lock-in: low** — MIT, pluggable checkpointer. Avoid LangSmith; you have Langfuse.
Mastra	ADOPT — PENDING THE KILL-TEST RE-RUN	**Verdict reversed** on reading the source. Apache-2.0; Convex store adapter ships; `requireApproval` gives `LOOP-5` granularity. Use its **Workflow** primitives, not its opaque agent loop, for Tier B/C — that keeps `LOOP-1` ("own the `while` loop... only if the loop stays readable") satisfied. Remaining risk: ~1 minor release/week, so pin the version in the manifest and upgrade on the eval suite, per `LOOP-9`.
Flue	PROBE, EXPECT THE SAME RESULT	Apache-2.0, 8.0k★, but created Feb 2026, **no tagged releases**, last push 8 Aug. Its "durable stream" is its own store. **Adoption risk: high** for a studio-wide standard.

COMPONENT	VERDICT	REASONING, LICENCE, INTEGRATION SURFACE, LOCK-IN
CrewAI	SKIP	Role/goal/backstory crews are the multi-agent-by-default pattern your own `TOOL-2` argues against. Adopting it would contradict your rules layer.

PHASE 4

Interoperability with studio-product-framework

Read: `AGENTS.md`, `agents/`, `docs/architecture.md`, `docs/production-readiness.md`, `packages/ai-runtime/src/*`, `packages/observability/src/*`, `packages/effect-critical/`, `apps/web/convex/schema.ts`.

There is no integration seam. There is one fork.

`studio-product-framework/agents/context/discovery/eval-first-spec.md` is a divergent copy of the `eval-first-spec` skill. Both are called the studio standard. They disagree on the pass bar.

	STUDIO-STANDARD-AGENT-FRAMEWORK	STUDIO-PRODUCT-FRAMEWORK
Golden cases	`20 REQUIRED` *"Fewer than 20 cases … is auto-fail."*	`"TARGET 20; MINIMUM 5 TO START COMMIT"`
Composition	Fixed spread ≥6 typical / ≥7 edge / ≥4 adversarial / ≥3 must-refuse	None
Reality floor	≥14 of 20 must be `[Fact]`	None
Mode coverage	Every failure mode reachable by ≥1 case	None
L1	"Drafts — proposes the output; human reviews and commits every one"	"Suggest + human confirm"
L2	**"Acts on approval** — prepares the action; human approves per item or batch"	**"Act in narrow band + audit"**
Acceptable rate	Derived: `tolerable_cost_per_cycle ÷ cost_of_one_failure`	Free-text table cell
Cost	`(C_attempt × A) + C_human + C_remediation`, gated against value	"Target cost per outcome (to the cent)"

L2 is not a wording difference — "acts on approval" and "acts in a narrow band with audit" are different autonomy semantics with different blast radii. **A fellow filling the product template clears the commit gate at five cases; the same artefact auto-fails the standard.**

> GOVERNANCE CONFLICT
>
> `studio-product-framework/docs/architecture.md`: *"Icarus is the discovery playbook — do not duplicate it here."* `studio-standard-agent-framework/README.md`: the Icarus modules are *"bundled here unchanged so the chain is testable in one place."* One repo forbids duplication; the other is built on it; and the product repo duplicated it anyway, divergently.

Contract mismatches, named

#	MISMATCH	STANDARD SAYS	PRODUCT FRAMEWORK HAS	SEVERITY
1	**Agent kind**	Never distinguishes them. "sandbox", "credit", "metering" appear **0 times**	*"Two kinds of agents… Do not conflate them in docs or APIs."* Runtime agents **must** sandbox — `run-agent.ts` hard-fails without `requireSandbox: true` — and meter credits	BLOCKING
2	**STATE-1 has no log to resume from**	*"resume from the Convex log alone"*	`schema.ts` has `users`, `subscriptions`, `webhookEvents`, `wallets`, `walletTransactions`, `inferenceRuns`. **No `agentRuns`. No `agentEvents`.** `inferenceRuns` is per-call, not per-run-event	BLOCKING
3	**Memory doctrine, three ways**	`MEM-1` / `MEM-7`: four tiers, Convex, tenant-keyed, indexed. `agent-design`: four Markdown files	**Hivemind (Deeplake)** for session/team memory; **git** for durable truth. A third store neither doc accounts for, and `MEM-8` demands queryability outside the writing tool	HIGH
4	**Tool definition type**	No schema specified	`gateway.ts` already exports `ToolDefinition` (name / description / JSON-Schema params) and `ToolExecutor`	MEDIUM — CHEAPEST FIX
5	Eval pass bar	20 cases hard	5 cases minimum to commit	HIGH
6	Runtime home vocabulary	`HOME-1`: Utopia OS / standalone Vercel / local	Convex + Clerk + Vercel. **"Utopia OS" appears nowhere in the product framework**	MEDIUM
7	Durable execution vendor	`STACK-2`: Inngest default, Trigger.dev for sovereign	Neither string appears in 291 files. Convex Workflow/Workpool components are what's referenced	MEDIUM
8	Surface model	Claude.ai / Claude Code / Cowork	*"Authoring ≠ runtime"* — these are authoring tools	MEDIUM
9	Autonomy ladder semantics	L0 Informs … L4 Autonomous	Different L1/L2 wording in the forked template	HIGH

What the product framework already gives you, unused

REAL, NAMED, AND CURRENTLY IGNORED BY THE STANDARD

- `packages/ai-runtime/src/types.ts` — `AgentRunRequest` (`runId`, `userId`, `goal`, `turns`, `requireSandbox`, `maxTurns`, `maxSpendCredits`), `AgentRunStatus`, `AgentRunResult`. **This is your run contract. It exists.**
- `budget.ts` — `assertWithinBudget()` enforces `LOOP-3`'s budget guard in code with **no default cap**. That is `LOOP-8` ("cap fan-out in code, not in a prompt") already implemented.
- `gateway.ts` — `ToolDefinition`, `ToolExecutor`, `ToolConfig`.
- `observability/src/events.ts` — `StudioEvents` already contains `agentRunStarted`, `agentRunSucceeded`, `agentRunFailed`, `inferenceCompleted`. **The event vocabulary exists.**
- `observability/src/langfuse.ts` — `STACK-4`, implemented, no-ops without keys.
- `effect-critical/` — `wallet.ts`, `meteredInference.ts`, with tests. Cost-per-outcome has a real ledger to measure against.
- `schema.ts` — `walletTransactions` carries a **`by_idempotency` index**. `LOOP-6` is already enforced for money.

And what is dead. `startAgentRun` is exported from `@studio/ai-runtime` and **called from nowhere** — I grepped every `.ts` / `.tsx` . `agentRunStarted` / `Succeeded` / `Failed` are defined and **emitted nowhere.** `run-agent.ts` validates, budget-gates, opens a sandbox, returns `status: "running"` — and stops, deferring the loop in its own comment. `gateway.ts` says *"One round only… No multi-hop agent loop."*

There is no agent loop anywhere in the studio. Not in the standard, not in the product framework.

The minimum interface set for clean modularity

Five contracts. Everything else can stay prose.

> `1 — AGENTMANIFEST`
>
> The machine-readable output of the chain, one file per agent, versioned in git. All required: `kind: "builder" | "runtime"` · `rung` · `tier` · `runtimeHome` · `talkSurface` · `toolIdentity` · `autonomy` · `evalContractRef` · `memoryTiers` (4 keys, `"unused"` permitted) · `waivers[]` . **This is the missing artefact.** Today the chain's output is a `.docx` — unparseable, ungateable, undiffable. Home: `schema/agent-manifest.schema.json` in the standard repo, consumed by the product framework's `commit-v1` loop.
>
> `2 — ADOPT THE EXISTING TOOL TYPE`
>
> `agent-design` Step 2's tool table gains a `schemaRef` column pointing at a `ToolDefinition` from `@studio/ai-runtime` . Mismatch #4 closes for roughly zero effort.
>
> `3 — AGENTRUNS + AGENTEVENTS CONVEX TABLES`
>
> Tenant-keyed and indexed, in `apps/web/convex/schema.ts` . Minimum: `agentRuns(runId, userId, manifestRef, status, startedAt, endedAt)` indexed `by_user` and `by_run` ; `agentEvents(runId, seq, type, payload, createdAt)` indexed `by_run_seq` , append-only, never mutated. Without these the entire "STATE-1 kill-test" section of the README describes a test with no target.
>
> `4 — ONE EVAL CONTRACT, ONE FILE, ONE BAR`
>
> Delete the product-framework template and point at the skill, or vice versa. Whichever survives, the number is stated once. Closes mismatches #5 and #9.
>
> `5 — EMIT THE EVENTS THAT ALREADY EXIST`
>
> `startAgentRun` writes `StudioEvents.agentRunStarted` to `agentEvents` and to Langfuse. That one wire turns `STACK-4` from an assertion into a fact and gives the framework its first observability of itself.

PHASE 5

Scoring

Overall — 4 / 10

A genuinely excellent rules document and PRD interview, packaged as an unversioned, unlicensed, untestable folder of Markdown with three internal contradictions, nine dangling dependencies, and a live fork against the repo it is meant to interoperate with. **The content would score 8. The container scores 2.** You asked for a framework; this is a very good draft of the specification half of one, and none of the runtime half.

Architecture	5	Coherent 3-layer intent; three unreconciled layer violations; one file is 34% of the corpus
Agent abstraction	3	Good spec shape, three competing taxonomies, misses the builder/runtime distinction the product framework calls binding
Modularity	3	*"Install all six. They are a chain, not a menu"* — anti-modular by its own statement; no package, no version, no import
Extensibility	3	Adding a stage touches 7–10 files, five shared. Adding a *rule* is excellent — that part alone is a 9
Observability	2	Mandates Langfuse from commit one; ships zero tracing of itself. Evidence base is screenshots
Testing	4	Real rubrics, real adversarial cases, honest judge protocol — all manual, 2 of 6 skills unharnessed, identical rubrics, known failures excluded from the headline
Documentation	7	Best dimension. Clear, honest about the undecided harness, S1–S7 is a real acceptance test. Loses points for the Hermes claim, 9 dangling refs, no licence
Production readiness	2	Nothing runs. `STATE-1`'s kill-test has no target schema to run against

SOTA competitiveness	3	On the runtime axis, not competitive — it isn't a runtime. On the governance axis it is genuinely ahead, and that axis isn't in the table

Remediation for every score below 7

ARCHITECTURE — 5

> WHAT
>
> Make `atelier-learnings` mechanically supreme instead of rhetorically supreme.
>
> WHERE
>
> `agent-design/SKILL.md` Step 3 · `workflow-design/SKILL.md` Step 5 · `agent-prd/SKILL.md` hard-gate line "At least ten eval tasks" + Appendix C's "Working / Episodic / Compounding".
>
> WHY
>
> A fellow can follow stage 2 exactly and violate `MEM-3`, `MEM-7`, `MEM-8` and `STACK-1` at once, and pass every gate.
>
> HOW
>
> Replace Step 3's four-store table with a rung-conditional one — rungs 1–3 use the Markdown stores; rung 4 uses `agentEvents` (episodic), a tenant-keyed `semanticFacts` table, `skills/` in git, assembled-per-call — plus the line *"At rung 4, a Markdown store fails MEM-7."* Retitle `workflow-design` Step 5's column **"Authoring surface"** and add a required adjacent **"Runtime home (HOME-1) — a different answer."** Change `agent-prd`'s gate to *"the eval contract from stage 3, at its own bar"*, and fix Appendix C to name all four tiers.

AGENT ABSTRACTION — 3

> WHAT
>
> Add `kind: builder | runtime` as a required Gate 0 field with kind-conditional gates.
>
> WHERE
>
> `agent-prd` Gate 0 "Implementation surface" + hard-gate checklist; `agent-builder` Step 0b.
>
> WHY
>
> Without it the chain emits PRDs for user-facing agents with no sandbox and no spend ceiling — the two things `@studio/ai-runtime` hard-fails without.
>
> HOW
>
> Ask it as intake 5b. When `runtime`, three mandatory hard-gate rows: sandbox provider named; `maxTurns` and `maxSpendCredits` stated as numbers (they map 1:1 onto `AgentBudget`); tenancy boundary named with its enforcing check. Then publish the tier↔rung mapping as a 4×3 table in Appendix B so the taxonomies stop floating free.

MODULARITY — 3

> WHAT
>
> Give the pack a package identity and a version.
>
> WHERE
>
> Repo root — `VERSION`, `CHANGELOG.md`, `LICENSE`; `version:` in all six frontmatters.
>
> WHY
>
> "Set in stone" with no version means no one can say which stone. A fellow who installed on 17 Aug and one who installs today are running different rules with no way to tell.
>
> HOW
>
> Semver the pack (`1.0.0`), stamp `pack_version` on each SKILL.md, tag the repo, add one line to Step 5's chain check: *"state the pack version in the deliverable."* Then decide distribution — a git submodule into `studio-product-framework`, or a published bundle. Manual upload of six folders to Claude.ai Personal Skills is not a distribution mechanism.

EXTENSIBILITY — 3

> WHAT
>
> Extract the five things every stage restates into one included set.
>
> WHERE
>
> New `_shared/` — `memory-table.md`, `evidence-ladder.md`, `kill-line.md`, `rung-ladder.md`, `gen-not-eval.md`.
>
> WHY
>
> The evidence ladder is copied verbatim into three skills; the memory table into three; gen≠eval into four. Every copy is a drift point, and Step 5 exists solely to re-reconcile drift the structure creates.
>
> HOW
>
> One file per shared concept; each skill links rather than copies. Step 5 then shrinks from "reconcile three vocabularies" to "assert the shared files were loaded."

OBSERVABILITY — 2

> WHAT
>
> Instrument the chain itself.
>
> WHERE
>
> The `AgentManifest`, plus a `runs/` log in this repo.
>
> WHY
>
> You cannot run `LOOP-9`'s scaffolding-removal experiment, or the bare-model deprecation run, or know your abandonment stage, without a record of runs. Today the only artefact of a chain run is a `.docx` and a memory.
>
> HOW
>
> Every completed chain emits a manifest JSON committed to `runs/YYYY-MM-DD-<slug>.json` carrying pack version, model, rung, tier, gates fired, blockers fired, stage reached. Ten of those make the first honest statement about whether the pipeline works.

TESTING — 4

> WHAT
>
> (a) correct the headline; (b) build a runner; (c) cover the two unharnessed skills; (d) differentiate the rubrics.
>
> WHERE
>
> `README.md` "Round 1" line · new `harness/run.ts` · new `agent-prd/tests/` and `atelier-learnings/tests/` · the three identical `rubric.json`.
>
> WHY
>
> The README overstates the evidence: A3 and A4 are documented failures excluded from the scored set while `gate_integrity` scored 5/5. In a pack whose first principle is that nothing grades its own work, that is the most damaging possible defect — and a five-minute fix.
>
> HOW
>
> **(a)** Change the line to *"6 golden pass · 1 partial · 2 adversarial failing (A3, A4) · 2 golden unscored (G8, G9)"* and add the adversarial rows to `RESULTS.md`. **(b)** `harness/run.ts`: read a case file, call the API with the skill(s) as system content, dump the transcript, then a second call with `rubric.json` + case + transcript as the judge, emitting scored JSON — the generator≠evaluator protocol, automated, making the "twice on every model release" cadence realistic. **(c)** `atelier-learnings` is testable as a *detector*: feed it designs violating `LOOP-3`, `CTX-6`, `MEM-7` and assert the right rule ID is cited. **(d)** Rename each rubric's dimensions to that skill's own promises, per Hermes' own standard.

PRODUCTION READINESS — 2

> WHAT
>
> Make the `STATE-1` kill-test runnable.
>
> WHERE
>
> `apps/web/convex/schema.ts` · `packages/ai-runtime/src/run-agent.ts` · a probe repo for the bake-off.
>
> WHY
>
> The README devotes a section to *"prove it by killing the process"* against a log that exists in no studio system. It cannot currently be run at all — which means neither Mastra's failure nor Flue's eventual result is measured against a real target.
>
> HOW
>
> Add the two tables. Wire `startAgentRun` to append `run.started`. Then run the bake-off against a real Convex log, with the `STATE-1` adapter clause decided first — otherwise every candidate fails by construction.

SOTA COMPETITIVENESS — 3

> WHAT
>
> Stop competing on the runtime axis. Declare the layer; adopt underneath it.
>
> WHERE
>
> `README.md` opening paragraph · `agent-prd` Appendix C.
>
> WHY
>
> Framing this as "our agent framework" invites a comparison it loses on eight of nine dimensions and obscures the one it wins on. Positioned as *the governance and eval layer over an adopted runtime*, it is genuinely ahead of all eight comparators.
>
> HOW
>
> Rewrite the first paragraph as *"the studio's specification, gating and eval standard for agents — runtime-agnostic, with the harness chosen per TOOL-3's fit gates."* Add `@convex-dev/agent` to the bake-off table and add the `STATE-1` adapter clause so the table can reach a verdict.

PHASE 6

Decision list

Ordered by impact-to-effort. Effort is engineer-days for one competent engineer.

#	RECOMMENDATION	IMPACT	DAYS	RISK IF SKIPPED	Y/N
1	Correct the Hermes headline in `README.md`; add the 5 adversarial rows (incl. A3/A4 failing) to `hermes/RESULTS.md`	HIGH	0.25	A pack built on "nothing grades its own work" publishes a self-scored result that hides its two known failures. Credibility damage on first careful read.	☐

#	RECOMMENDATION	IMPACT	DAYS	RISK IF SKIPPED	Y/N
2	Add `LICENSE` to `studio-standard-agent-framework`	HIGH	0.25	Public repo, no licence = all rights reserved. Nobody may legally reuse it, including other Utopia entities and clients.	☐
3	Fix the eval-bar contradiction — `agent-prd` 's hard gate cites stage 3's contract instead of restating "ten"	HIGH	0.5	The last gate before work orders is looser than the kill line three stages earlier. Gate inversion.	☐
4	Add the `STATE-1` adapter clause (or declare `STATE-1` absolute) *before* any further bake-off probe	VERY HIGH	1	As written, `STATE-1` disqualifies every harness with its own store. The bake-off cannot terminate; Flue's probe will "fail" for a reason that has nothing to do with Flue.	☐
5	**Re-run the `STATE-1` kill-test against Mastra + `@mastra/convex`** (the first probe used the default LibSQL store). Keep `@convex-dev/agent` as the fallback candidate	VERY HIGH	0.5	The bake-off is currently blocked on a result that measured a configuration, not the framework. Half a day closes it either way.	☐
6	Resolve the fork — one `eval-first-spec` , one pass bar, one autonomy ladder	VERY HIGH	1	Two "studio standards" with different bars (5 vs 20 cases) and different L2 semantics, in two repos, already live.	☐
7	Add `kind: builder | runtime` as a required Gate 0 field, with sandbox + `maxTurns` + `maxSpendCredits` mandatory when runtime	VERY HIGH	1	The chain emits PRDs for user-facing agents with no sandbox and no spend ceiling — the two things `@studio/ai-runtime` hard-fails without.	☐
8	Rung-condition the memory model in `agent-design` Step 3; fix Appendix C's three-name/four-tier error	HIGH	1	Following stage 2 exactly violates `MEM-3` , `MEM-7` , `MEM-8` and `STACK-1` at once, and passes every gate.	☐
9	Resolve the 9 dangling skill references — vendor, or mark external with a URL and a "what to do if you don't have it" line	HIGH	1	Two of them are *blocker exits*. The chain stops and routes a fellow to a skill they do not have.	☐
10	Retitle `workflow-design` Step 5 to "Authoring surface"; add a required adjacent `HOME-1` runtime-home field	MEDIUM	0.5	Stage 1 hands stage 4 a surface built from a vocabulary the rules layer classifies as authoring-only.	☐
11	Version the pack — `VERSION` + `CHANGELOG.md` + `pack_version` frontmatter + a git tag	HIGH	1	"Set in stone" with no version. Two fellows run different rules with no way to detect it.	☐
12	Define `AgentManifest` JSON Schema as the chain's machine-readable output, alongside the `.docx`	VERY HIGH	3	The chain's only output is an unparseable document. Nothing downstream can gate on it, diff it, or count it. **The keystone interop artefact.**	☐
13	Add `agentRuns` + `agentEvents` to `schema.ts` ; emit `agentRunStarted` from `startAgentRun`	VERY HIGH	4	`STATE-1` 's kill-test has no target. `STACK-4` is an assertion, not a fact. `startAgentRun` and the `agentRun*` events stay dead code.	☐
14	Build `harness/run.ts` — automated case runner + separate-judge scorer emitting scored JSON	HIGH	5	26 manual cases across 4 harnesses means the "full set on every edit, twice on every model release" cadence will not happen.	☐
15	Extract `_shared/` (memory table, evidence ladder, kill line, rung ladder, gen≠eval) and link instead of restating	MEDIUM	2	Five concepts copied across 3–4 files each. Every copy is a drift point; Step 5 exists only to re-reconcile drift the structure creates.	☐

Total if all fifteen: ~21 engineer-days (~24 including the item-5 probe). Items 1–3 are one day combined and I would do them today.

What I'd deep-dive on request, and what I'd need

DEEP DIVE	WHAT I'D NEED FROM YOU
The `STATE-1` adapter clause, written as a rule with a conformance test	Confirmation of (A) adapter clause vs (B) absolute. If (A): permission to prototype a Convex checkpointer against LangGraph's `libs/checkpoint-conformance` suite
`@convex-dev/agent` probe against `STATE-1`	A throwaway Convex deployment I may write to, plus your W-01 waiver format if the probe log can't live in production

DEEP DIVE	WHAT I'D NEED FROM YOU
The `AgentManifest` schema, drafted	Which side owns it — I'd argue the standard repo, consumed by `commit-v1` — and whether the `.docx` stays primary for non-engineers
`harness/run.ts` built, and the current 26 cases run through it	An API key with a budget I may spend, and a decision on whether transcripts are committed or gitignored
Full read of the 9 missing Icarus skills for contradictions with `atelier-learnings`	Access to the Icarus pack — referenced by module number in five places, and I have not read any of it
Whether `agent-prd` should be split	1,435 lines is beyond what a model reliably holds. I'd want 3–5 real chain transcripts to see where stage-4 adherence degrades — Hermes `A2` suggests it does
Product-framework test-suite health	Permission to `pnpm install` and run `pnpm test` / `typecheck`. I did **not** run them; the 4 test files are reported as present, not as passing

Placeholders you left unfilled, and what I assumed

PLACEHOLDER	ASSUMED	CHANGE IT IF WRONG
Branch	`main` — default on both	—
Clone target	`/Users/kp/Studio Agent Framework/` — was empty	—
Primary language / runtime	**TypeScript**, from the product framework (317 KB TS) and every stack decision in Appendix C. The standard repo has no language	If Python is in play, LangGraph's position improves materially
Deployment context	Convex + Clerk + Vercel, from `docs/architecture.md`	—
Team size / skill level	**2 engineers + non-technical fellows** — inferred from 2 contributors on the product repo, 1 on the standard, and the rung ladder's explicit non-builder path	**The assumption most likely to be wrong**, and it drives items 12–14. A 2-person team cannot maintain a six-document pack *and* a bake-off *and* a runner without cutting something
Diagram output format	Mermaid in Markdown, plus this page	Say the word for rendered SVG
Diagram save path	`audit/architecture.md` in the working directory — **not** inside either repo, per your no-writes rule	—

Nothing was written, committed, or pushed to either repository. Both were cloned read-only and left untouched.